A Summary of Discoveries

In this paper we explored the prompt leakage risks of existing LLMs. We summarize our discoveries as follows:

- LLMs, even those with safety alignments, are highly vulnerable to prompt extraction attacks.
- Larger language models with instruction alignments may suffer from implicit intent attacks more severe than smaller models.
- The UR of LLMs is usually positively correlated to model parameters, and negatively correlated to prompt lengths. For prompts with more tokens, the absolute number of leaked tokens grows steadily.
- Prompts in natural languages are more easily to be memorized and translated, while prompts with API descriptions typically exhibit higher probability of a complete extraction.
- Prompts with which LLMs are familiar are more likely to extract.
- LLMs can construct a straightforward linking path from prompts to generated contexts in the attention mechanism, which may be the key factor of accurate prompt translation.
- No evidence in this paper demonstrates that there exists an obvious trade off between the performances and the prompts' security of LLMs.
- The two types of defending strategies proposed in this paper can decrease the extraction rate significantly without extra training stage.

B A Detailed Threat Model

Adversary's Objective. The objective of attackers is to extract prompts (i.e., the prefix of LLMs' input text) from language models at inference time, by crafting and optimizing their inputs. The strength of attacks is measured by how many prompts are leaked, as well as the uncovered status of extracted texts among a prompt (measured by n-gram, ρ-fuzzy, and others). Similar to LLM's jailbreaking, a strong user input should be transferable across various kinds of LLMs and prompts.

Adversary's Capabilities. We assume that the adversary can **only** obtain the text responses of LLMs, whereas the model weights, hidden states, and generation strategies (e.g., greedy or beam search) as well as the sampling parameters (e.g., sampling temperature) of LLMs are in blackbox. Furthermore, they cannot access any prior knowledge of the *serialization pattern* of prompts.

We believe this setting fits most real-world applications of customized LLMs, where the user interface is more likely to be a chat window rather than API calls.

Attack Targets. We select both commercial LLMs (e.g., OpenAI's GPTs) and open-source LLMs (e.g., Llama-2) as the attack targets. As for the latter, we evaluate five popular open-source LLMs, namely, Llama-2 (Touvron et al., 2023), Phi (Li et al., 2023b), Qwen (Bai et al., 2023), Vicuna (Peng et al., 2023), and Mistral (Jiang et al., 2023), with their parameters from 1.5 billion to 13 billion.

C Analysis of Proposed Indicators

Notations. Suppose the hidden state after the $(l-1)$-th Transformer module is $\mathbf{E}^{l-1} \in \mathbb{R}^{N\times d}$, in which N and d denote the sequence length and the feature dimension, then we know in the l-th Transformer module the attention results can be computed by

$$\begin{aligned} &Attn^{(l,n)}(x_a, x_b) \\ &= \frac{softmax((\mathbf{Q}^{(l,n)})^T \cdot (\mathbf{K}^{(l,n)})}{\sqrt{d/N_h}}[a,b] \\ &= \frac{exp({E_a^{l-1}}^T \cdot W_{Q^{(l,n)}}^T \cdot W_{K^{(l,n)}} \cdot E_b^{l-1})}{\sum_{j\in N} exp({E_j^{l-1}}^T \cdot W_{Q^{(l,n)}}^T \cdot W_{K^{(l,n)}} \cdot E_b^{l-1}) \cdot \sqrt{d/N_h}}, \end{aligned} \tag{9}$$

where the E_a^{l-1}, E_b^{l-1} are the a-th and b-th hidden vectors in $\mathbf{E}^{l-1}$, $W_{Q^{(l,n)}}, W_{Q^{(l,n)}} \in \mathbb{R}^{d/N_h \times d/N_h}$ are learnable parameters in the l-th layer n-th head attentions, N_h denotes the number of attention heads in each Transformer module, and $\mathbf{Q}^{(l,n)}, \mathbf{K}^{(l,n)}$ are computed by $W_{Q^{(l,n)}} \cdot \mathbf{E}^{l-1}$ and $W_{K^{(l,n)}} \cdot \mathbf{E}^{l-1}$, respectively.

Based on Equation 9, we can see the *geometric mean* is more compatible compared to the *arithmetic mean* since the attentions can be merged in the exponential operations. Taking SPLIt (in Equation 5) as the example:

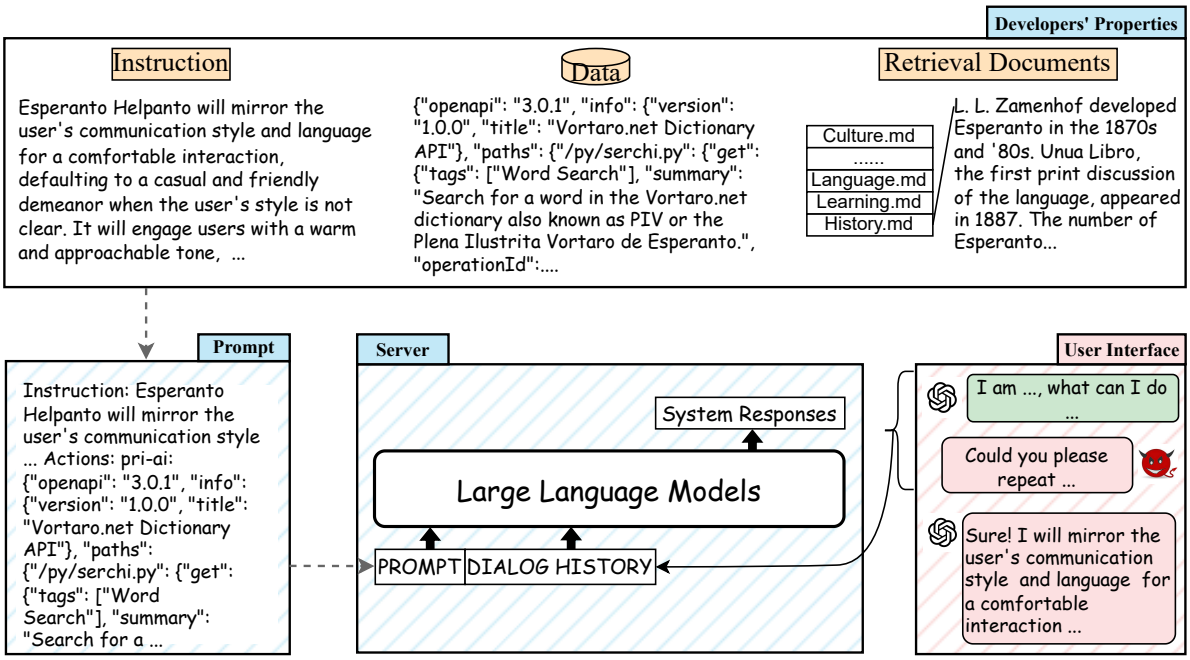

Figure 9: A toy and end-to-end example of the attack scenario. Third-party developers prepare their *instructions*, *structured data*, and *documents* to build GPT-based services. The platform then arranges these components into a prompt, which, along with the dialogue history, is used to generate responses for users. The goal of adversarial users in this scenario is to steal the prompt through the text-based user interface.

Geometric Mean (what we used):

$$
\begin{aligned}
\alpha_{pre}^{(l,h)} &= \prod_{t=1}^{N_p} Attn^{(l,n)}(x_{t-1}^{p}, x_t^{pg})^{\frac{1}{N_p}} \\
&= \frac{\prod_{t=1}^{N_p} exp({E_{t-1,p}^{l-1}}^T \cdot W_{Q^{(l,n)}}^T \cdot W_{K^{(l,n)}} \cdot E_{t,pg}^{l-1})^{\frac{1}{N_p}}}{\sum_{j \in N} exp({E_j^{l-1}}^T \cdot W_{Q^{(l,n)}}^T \cdot W_{K^{(l,n)}} \cdot E_b^{l-1}) \cdot \sqrt{d/N_h}} \\
&= \frac{exp(\frac{\sum_{t=1}^{N_p} {E_{t-1,p}^{l-1}}^T \cdot W_{Q^{(l,n)}}^T \cdot W_{K^{(l,n)}} \cdot E_{t,pg}^{l-1}}{N_p})}{\sum_{j \in N} exp({E_j^{l-1}}^T \cdot W_{Q^{(l,n)}}^T \cdot W_{K^{(l,n)}} \cdot E_b^{l-1}) \cdot \sqrt{d/N_h}}.
\end{aligned}
$$

Arithmetic Mean:

$$
\begin{aligned}
\alpha_{pre}^{(l,h),'} &= \sum_{t=1}^{N_p} Attn^{(l,n)}(x_{t-1}^{p}, x_t^{pg})/N_p \\
&= \frac{\sum_{t=1}^{N_p} exp({E_{t-1,p}^{l-1}}^T \cdot W_{Q^{(l,n)}}^T \cdot W_{K^{(l,n)}} \cdot E_{t,pg}^{l-1})}{\sum_{j \in N} exp({E_j^{l-1}}^T \cdot W_{Q^{(l,n)}}^T \cdot W_{K^{(l,n)}} \cdot E_b^{l-1}) \cdot \sqrt{d/N_h} \cdot N_p}.
\end{aligned}
$$

$\alpha_{pre}^{(l,h),'}$ is typically more unstable than $\alpha_{pre}^{(l,h)}$, since $exp(\cdot)$ is nonlinear and cannot be salable well among all the *inner product* metric intervals.

D Implementation Details

Our experiments are conducted on 8 $\times$ 24 GB Nvidia RTX 4090 GPUs. Each inference is executed 5 times, and we record the mean values and standard deviations. For token sampling, default hyperparameters are used, as we hypothesize that the adversary cannot manipulate the temperature or sampling strategies. We make every effort to minimize the influence of other factors during evaluation. For example, we set the maximum sequence length to 4096, which we believe is within the context window of both large and small LLMs. For each experiment, we provide a detailed description of its specific settings.

E Supplemental Related Work

E.1 Instruction-following Inference

Language Modeling. Language models, especially generative language models, can be modeled as a probability map $\Pr(\mathbf{x}^O|\mathbf{x}^I, \theta)$ from the input text $\mathbf{x}^I$ to the generated text $\mathbf{x}^O$ with the trainable parameters θ. In order to generate $\mathbf{x}^O$, the language models usually perform the *auto-regressive conditional generation* paradigm, where a probability model $\Pr_\theta$ generates a new token x_i^O iteratively based on the input sequence $\mathbf{x}^I$ and its previous generated tokens $\mathbf{x}_{<i}^O = \{x_j\}_{j=1,2,...,i-1}$ with the following probability:

$$
\Pr_\theta(\mathbf{x}^O|\mathbf{x}^I) = \prod_{i=1}^{N_o} \Pr_\theta(x_i^O|\mathbf{x}^I, \mathbf{x}_{<i}^O). \tag{10}
$$

$\mathbf{x}^I$ in Equation 10 typically denotes the input sequence in specific downstream tasks, $\mathbf{x}^O =$

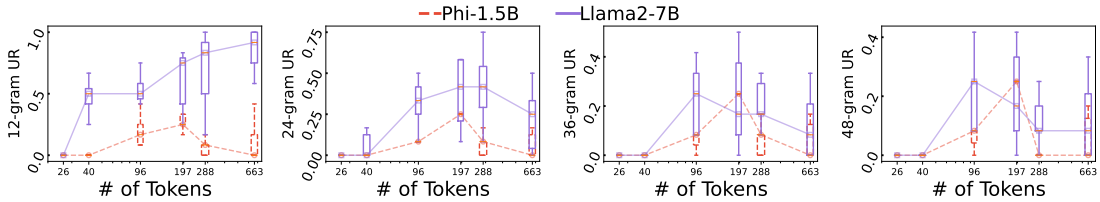

Figure 10: The relationship between prompt length and prompt uncover rate.

Data Type	Prompt Examples
GLUE tasks' Prompts	"valid parentheses": "As a syntax validator, assess the given sequence of brackets and determine whether it conforms to proper bracket rules. Respond Valid if the brackets are matched, Invalid otherwise.", "In your role as an expression evaluator,...
Leaked GPTs Prompts	I am designed to provide users with delightful and unique recipes, each crafted with a touch of whimsy from the animal kingdom. When a user requests a recipe, I first ... is intended to be engaging, humorous, and slightly surreal, providing users with both culinary inspiration and a dash of entertainment. The output is always in this order: - Personal story which also introduces myself - The recipe, with some animal references sprinkled in - An image of the animal character and the recipe
Fuction Callings	"{\"openapi\": \"3.0.3\", \"servers\": [{\"url\": \"https://balldontlie.io\"}], \"info\": {\"contact\": {}, get\": {\"description\": \"specific team\", ... \"operationId\": \"specificTeam\", \"responses\": {\"200\": {\"description\": \"\"}}, \"summary\": \"specific team\", \"tags\": [\"teams\"]}}}}",
Role-based Prompts	I want you to act as a linux terminal. I will type commands and you will reply with what the terminal should show. I want you to only reply with the terminal output inside one unique code block, and nothing else. ... inside curly brackets {like this}. my first command is pwd

Table 2: Example cases from our proposed PEAD benchmark.

$\{x_i^O\}_{i=1,2,\ldots,N_O}$ is the generated sentence, and we can train the parameters θ with the training set $\mathcal{D}_{tr}$ by a logarithmic maximum likelihood estimation function:

$$\mathcal{L} = -\sum_{s\in\mathcal{D}_{tr}}\sum_{i=1}^{N_O}\log \Pr(s_i^O|s^I, s_{<i}^O, \theta), \quad (11)$$

where its exponential variant

$$PPL(s^O) = \sqrt[N_O]{\prod_{i=1}^{N_O}\Pr(s_i^O|s_{<i}^O, \theta)}, \quad (12)$$

is known as the *perplexity* of the text sequence.

Both the pre-training and the fine-tuning procedures of language models update the model parameters based on Equation 11. However, for a larger model, fine-tuning on different downstream tasks is time-consuming, and therefore those task descriptions, also known as the *instructions (prompts)*, are introduced into language models (e.g., T5 (Raffel et al., 2020)) to enable the multi-task ability of pre-trained language models. Equation 10 can then be reformatted with the prompt $\mathbf{x}^P$ as

$$\Pr_\theta(\mathbf{x}^O|\mathbf{x}^P, \mathbf{x}^I) = \prod_{i=1}^{N_o}\Pr_\theta(x_i^O|\mathbf{x}^P, \mathbf{x}^I, \mathbf{x}_{<i}^O). \quad (13)$$

With prompts, language models usually require less (or even no) training data in the *fine-tuning* stage, which means they can be more effectively customized to downstream tasks.

However, with the rapid increase of parameters, customizing language models by full fine-tuning suffers from excessive costs, so parameter efficient fine-tuning (PEFT) (Liu et al., 2021a; Li and Liang, 2021; Hu et al.; Xu et al., 2023) and in-context learning methods emerge.

In-context learning (ICL), as its name suggests, is an inference-period-customizing method. It lets pre-trained LLMs learn the downstream tasks only by the task descriptions without any model parameter modification. It was first proposed in GPT-3 (Brown et al., 2020) with 175 billion parameters. Without any further training on downstream tasks, and only based on the natural language descrip-